Day 2 - OOPs Inheritance

ITSRUNTYM

What is Inheritance?

Inheritance is one of the four pillars of **Object-Oriented Programming (OOP)** (along with Abstraction, Encapsulation, and Polymorphism).
Definition: Inheritance is the process where one class **acquires the properties and behaviors (fields and methods)** of another class.

Why Use Inheritance?

- **Code Reusability**: Avoid code duplication
- **Logical Hierarchy**: Model real-world relationships (Car IS-A Vehicle)
- **Extensibility**: Easily add new features without modifying old code
- **Polymorphism**: Enables method overriding and dynamic dispatch

Disadvantages of Inheritance

- Tight coupling
- Overriding might break base functionality
- Can lead to fragile base class problem

Real-Life Analogy:

Imagine:

- Vehicle is a general category (base class).
- Car, Bike, and Bus are specific types of vehicles (subclasses).
  They all have **common features** like start(), stop() which should not be rewritten every time.

Basic Syntax:

```
class Parent {
   void display() {
```

```
        System.out.println("This is parent class");
    }
}
class Child extends Parent {
    void show() {
        System.out.println("This is child class");
    }
}

public class Test {
    public static void main(String[] args) {
        Child obj = new Child();
        obj.display(); // inherited method
        obj.show();    // child's own method
    }
}
```

What is Coupling?
Coupling refers to the **degree of dependency between classes or modules**.

- **Tight Coupling**: Classes are **highly dependent** on each other.
- **Loose Coupling**: Classes are **independent** and communicate through abstraction/interfaces.

Tight Coupling (BAD practice)
Definition:
A class is **tightly coupled** to another when it directly creates or depends heavily on that class's implementation.

Code Example:

```
class Engine {
    public void start() {
        System.out.println("Engine Started");
    }
}
class Car {
    Engine engine = new Engine();      // tight coupling

    public void startCar() {
        engine.start();
    }
```

```
}
```

Problem:

- Car is **dependent on the concrete class** Engine
- If we change the Engine class, Car must be changed too

Violates:

- Open/Closed Principle
- Dependency Inversion Principle

Loose Coupling (GOOD practice)
Definition:
Loose coupling means reducing dependencies between components so that changes in one do **not** affect others.

Achieved via:

- **Interfaces**
- **Dependency Injection (DI)**
- **Inversion of Control (IoC)**

Code Example:

```
interface Engine {
    void start();
}
class PetrolEngine implements Engine {
    public void start() {
        System.out.println("Petrol Engine Started");
    }
}
class Car {
    private Engine engine;

// Constructor Injection (Dependency Injection)

    Car(Engine engine) {
        this.engine = engine;
    }

    public void startCar() {
        engine.start();
```

```
  }
}
```

Benefits:

- You can replace PetrolEngine with DieselEngine, ElectricEngine, etc. without changing Car
- Follows **Open/Closed Principle**

Real-Life Analogy

Scenario	Description
Tight Coupling	A remote that only works with one specific brand of TV
Loose Coupling	A universal remote that can work with any TV via settings

Diagram

Tight Coupling

```
Car ————————> Engine
      (direct dependency)
```

Loose Coupling

```
Car ————————> Engine (interface)
        ▲
    PetrolEngine, DieselEngine
```

IS-A and HAS-A Relationships

Type	Meaning	Example
IS-A	Inheritance (extends / implements)	Dog IS-A Animal
HAS-A	Composition / Aggregation (fields)	Car HAS-A Engine

Types of Inheritance in Java

Type	Description
Single	One subclass inherits one superclass
Multilevel	A class inherits a subclass which itself inherits another class
Hierarchical	Multiple classes inherit from a single superclass
Multiple (Not supported directly)	Java doesn't support multiple inheritance with classes (only with interfaces)
Hybrid	Combination (achieved using interfaces)

1. Single Inheritance

One subclass inherits one superclass

```
class Animal {
    void eat() {
        System.out.println("Animal eats food");
    }
}
class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
```

```
}
```

Dog IS-A Animal

2. Multilevel Inheritance

A class inherits a subclass which itself inherits another class

```
class Animal {
    void eat() {
        System.out.println("Eating...");
    }
}
class Dog extends Animal {
    void bark() {
        System.out.println("Barking...");
    }
}
class Puppy extends Dog {
    void weep() {
        System.out.println("Weeping...");
    }
}
```

Puppy IS-A Dog, Dog IS-A Animal

3. Hierarchical Inheritance

Multiple classes inherit from a single superclass

```
class Animal {
    void eat() {
        System.out.println("Eating...");
    }
}
class Dog extends Animal {
    void bark() {
        System.out.println("Barking...");
    }
}
```

```
class Cat extends Animal {
    void meow() {
        System.out.println("Meowing...");
    }
}
```

Both Dog and Cat inherit from Animal

4. Multiple Inheritance (Via Interface)

```
interface Printable {
    void print();
}
interface Showable {
    void show();
}
class Document implements Printable, Showable {
    public void print() {
        System.out.println("Printing document");
    }
public void show() {
        System.out.println("Showing document");
    }
}
```

Document inherits from **two interfaces**

What is Multiple Inheritance?

Definition: Multiple inheritance is a feature where a **class can inherit from more than one class**.

Why Java Doesn't Support Multiple Inheritance with Classes

**Main reason: To avoid *ambiguity and complexity* in the inheritance chain.

Imagine:

```java
class A {
    void show() {
        System.out.println("A");
    }
}
class B {
    void show() {
        System.out.println("B");
    }
}
// This is not allowed
class C extends A, B {
    // Compilation Error: Class can't extend multiple classes
}
```

Java throws error: *"cannot inherit from multiple classes"*

Problem: If C.show() is called, Java can't decide whether to call A.show() or B.show().

This is called the **Diamond Problem**.

The Diamond Problem Explained (Visually)

```
    A
   / \
  B   C
   \ /
    D
```

If both B and C override a method from A, and D inherits from both, which version does D inherit?

Java's Solution: Use Interfaces for Multiple Inheritance

Example: Multiple Inheritance with Interfaces

```
interface A {
    default void show() {
        System.out.println("A");
    }
}

interface B {
    default void show() {
        System.out.println("B");
    }
}

//  Class can implement multiple interfaces

class C implements A, B {
    // Must override show() to resolve conflict

    public void show() {
        System.out.println("C chooses to call A:");
        A.super.show();  // explicitly calling A's version

        System.out.println("C chooses to call B:");
        B.super.show();  // explicitly calling B's version
    }
}

public class Main {
    public static void main(String[] args) {
        C obj = new C();
        obj.show();
    }
}
```

Output:

```
C chooses to call A:
A
C chooses to call B:
B
```

Here, you **explicitly resolve the conflict**, so Java is happy.

The super Keyword

Used in 3 ways:

1. **Access parent class constructor**
2. **Access parent class method**
3. **Access parent class field**

Example:

1. Access Parent Class Constructor

Usage:

Use super() to **call the constructor of the parent class** from the child class constructor.

```
class Animal {
    Animal() {
        System.out.println("Animal constructor called");
    }
}
class Dog extends Animal {
    Dog() {
        super(); // calls Animal's constructor
        System.out.println("Dog constructor called");
    }
}
public class Test {
    public static void main(String[] args) {
        Dog d = new Dog();
    }
}
```

Output:

```
Animal constructor called
Dog constructor called
```

Notes:

- super() must be the **first statement** in the constructor.
- If not written explicitly, Java compiler inserts it automatically **if parent**

has a default constructor.

2. Access Parent Class Method

Usage:
If a method is **overridden** in child class, you can use super.method() to call **parent's version**.

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}
class Dog extends Animal {
    void sound() {
        super.sound();            // call Animal's sound()
        System.out.println("Dog barks");
    }
}
public class Test {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();
    }
}
```

Output:

```
Animal makes sound
Dog barks
```

Why Use?

- To **reuse parent logic** and add extra behavior.
- Helps avoid duplicating code.

3. Access Parent Class Field (Variable)

Usage:
If a child class has a variable with the **same name as in parent**, use super.variable to refer to the **parent's version**.

```
class Animal {
    String color = "white";
}
```

```
class Dog extends Animal {
   String color = "black";
void printColor() {
      System.out.println("Dog color: " + color);        // black (current class)
      System.out.println("Animal color: " + super.color); // white (parent class)
   }
}
public class Test {
   public static void main(String[] args) {
      Dog d = new Dog();
      d.printColor();
   }
}
```

Output:

```
Dog color: black
Animal color: white
```

Interview Specific

When to Use Inheritance?

- **"IS-A" relationship exists** logically
- You want to **reuse** and **extend** behaviour
- Shared code for multiple subclasses
- For enabling **polymorphism**

Where Not to Use Inheritance?

- When **composition** is better: Use HAS-A instead
- If it leads to **tight coupling**
- If child modifies too much of parent behaviour

Real World Example: Banking System

Classes:

- Account (parent): balance, deposit(), withdraw()
- SavingsAccount, CurrentAccount (child): different interest, minimum balance

```
class Account {
   protected double balance = 0;
public void deposit(double amt) {
      balance += amt;
   }
public void withdraw(double amt) {
      balance -= amt;
   }
}
class SavingsAccount extends Account {
   private double interestRate = 0.05;
public void addInterest() {
      balance += balance * interestRate;
   }
}
```

SavingsAccount IS-A Account

Advantages of Inheritance

- Reusability
- Easier to maintain
- Polymorphism support
- Logical structure of hierarchy

Disadvantages of Inheritance

- Tight coupling
- Overriding might break base functionality
- Can lead to fragile base class problem

Inheritance vs Composition

Aspect	Inheritance	Composition
Relationship	IS-A	HAS-A
Coupling	More tightly coupled	Loosely coupled
Flexibility	Less	More
Example	Car extends Vehicle	Car has Engine

Summary

Concept	Purpose	Syntax/Example

Inheritance	Reuse code from another class	class B extends A {}
Types	Single, Multilevel, Hierarchical, Multiple (via interface)	See above
super	Access parent constructor/method/ variable	super(), super.method(), super.variable
IS-A	Inheritance relationship	Dog IS-A Animal
HAS-A	Composition/ aggregation	Car HAS-A Engine